MCMP117 · WEEK 6

Fundamentals of Operating Systems

Week 6 · Memory Management: Contiguous Allocation, Paging & Segmentation

Many programs, one RAM: how does every process get its own safe piece of memory?

Electronical Communication Technology Programme · Vocational School · Istanbul Okan University

LAST WEEK

Week 5 in Four Ideas

Scheduling algorithms

FCFS, SJF/SRTF, Priority and Round Robin, compared with Gantt charts and average waiting time.

The time quantum

Round Robin with a tiny quantum wastes time on context switches; a huge one turns into FCFS.

Race conditions

When threads update shared data without coordination, the result depends on timing. The critical section must be protected.

Mutexes & semaphores

lock()/unlock() and wait()/signal() give mutual exclusion and let threads wait without burning the CPU.

Week 5 shared the CPU in time. Today we share main memory in space: many processes in RAM at once, each protected from the others.

Today's Agenda

1. **Basics** Address binding, logical vs. physical addresses, the MMU, base & limit registers, swapping.
2. **Contiguous allocation** Holes, first-fit / best-fit / worst-fit, fragmentation, compaction.
3. **Paging** Pages and frames, page tables, address translation, the TLB, effective access time.
4. **Segmentation** The programmer's view of memory, segment tables — and what x86-64 Linux really does.
5. **Hands-on** Real addresses on Linux: /proc/self/maps, fork() and the same address twice.

PART 1

Memory Management Basics

Many processes in one RAM — and none may touch another's memory.

MOTIVATION

Why Does Memory Need Managing?

The CPU can only execute instructions and read data that are in main memory (registers and cache are copies of RAM).

Many processes at once

- Multiprogramming (Week 1) keeps several processes in RAM so the CPU is never idle.
- Who gets which part of RAM?

Protection

- A process must not read or overwrite the memory of another process — or of the kernel.
- Checking every access in software is far too slow: the hardware must do it.

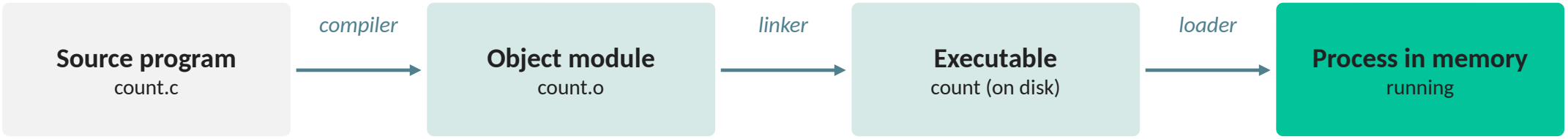
Relocation

- A program does not know in advance where in RAM it will be loaded.
- It may even be moved while it runs.

Reminder: every process has the layout text · data · heap · stack (Week 3). Today: where do these bytes actually live in RAM?

When Does a Name Become an Address?

A program refers to variables by name; the hardware needs numbers. Binding can happen at three different times.



Compile time

If the final location is known, the compiler can emit absolute addresses. Moving the program means recompiling. (MS-DOS .COM files)

Load time

The compiler emits relocatable code ("14 bytes from the start"); the loader fixes the addresses. Once loaded, the program cannot move.

Execution time

Addresses are translated by hardware on every access, so the process can be moved while it runs. Used by all modern OSs.

Logical vs. Physical Addresses

Logical (virtual) address

- Generated by the CPU while the program runs.
- What the program — and you, with printf("%p") — sees.
- Every process can start at address 0.
- **Logical address space:** the set of all logical addresses of a process.
- Example: 346 in the MMU example on the next slide; 0x55f2… printed by our C program.

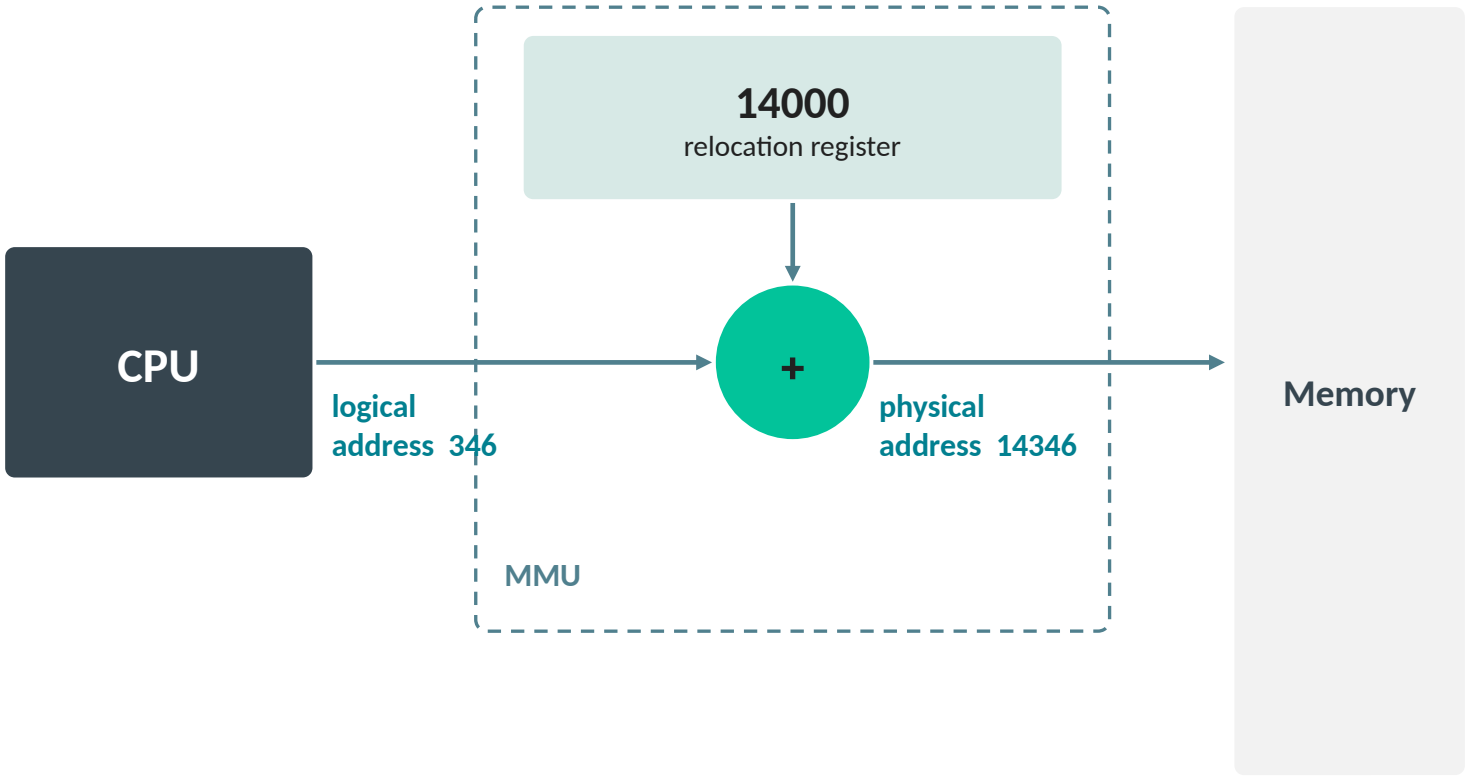
Physical address

- The address that is actually put on the memory bus.
- Seen only by the memory hardware.
- Each byte of RAM has exactly one physical address.
- **Physical address space:** the set of physical addresses that correspond to them.
- Example: 14346 — where logical 346 really ends up in RAM.

With compile- or load-time binding the two are identical. With execution-time binding they differ, and a hardware unit — the memory-management unit (MMU) — translates one into the other.

The MMU with a Relocation Register

The simplest MMU adds the value of a relocation (base) register to every address the CPU generates.

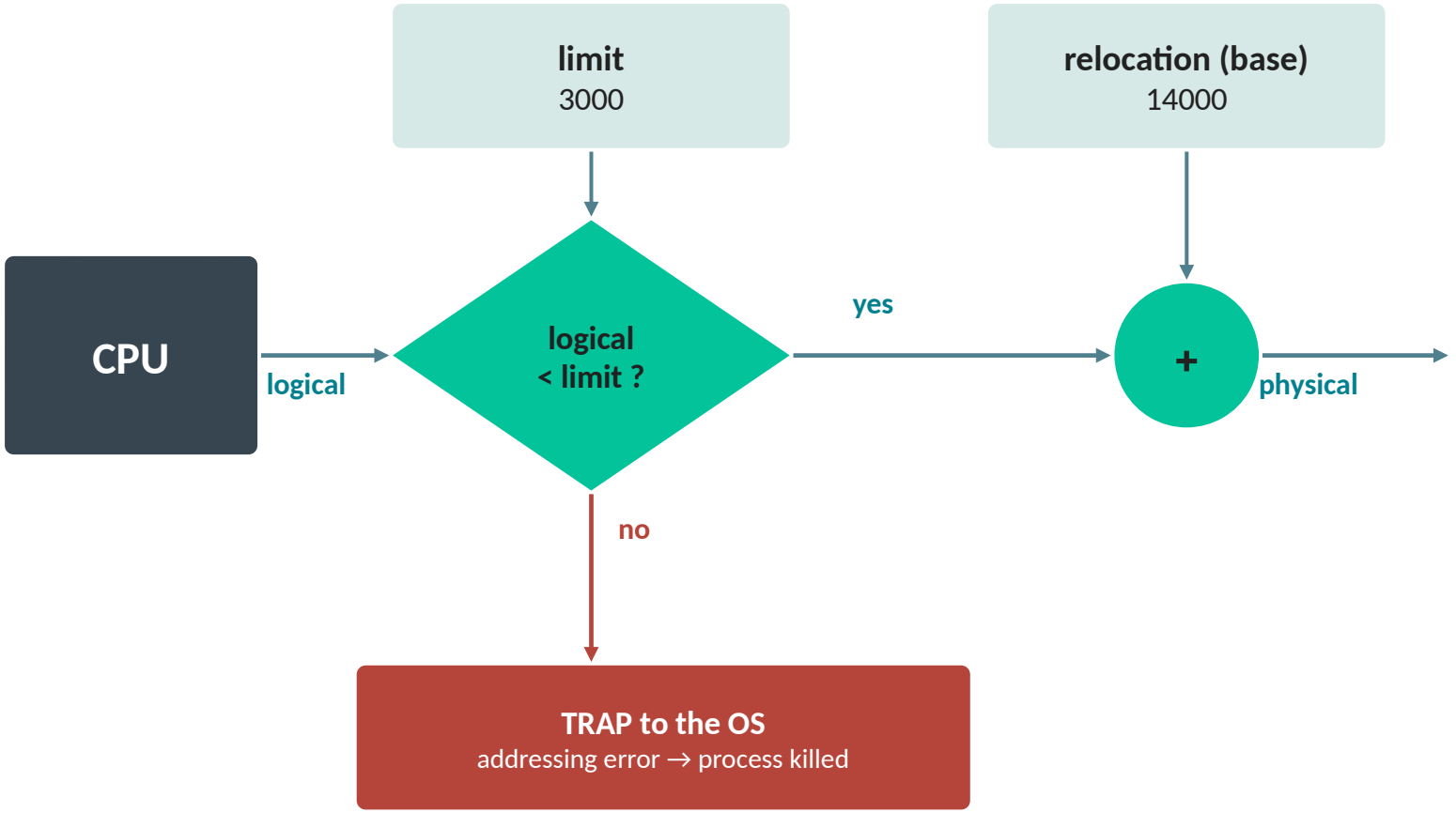


What it means

- The program believes it runs at addresses 0 … 2999.
- The OS loaded it at 14000 and wrote 14000 into the relocation register.
- To move the process, the OS copies it and changes one register.
- Loading the register is a privileged instruction: only the kernel may do it (Week 2).

Base and Limit Registers: The Protection Check

A second register, the limit, holds the size of the process. Every access is checked before relocation.



Rules

- Legal: 0 ≤ logical < limit.
- Physical = base + logical.
- Only the kernel can load these registers.
- On Linux the trap appears as "Segmentation fault".

WORKED EXAMPLE

Base & Limit in Decimal and in Hex

Base	Limit	Logical address	Check: logical < limit?	Result
14000	3000	346	346 < 3000 ✓	physical 14346
14000	3000	2999	2999 < 3000 ✓	physical 16999 (last legal byte)
14000	3000	3500	3500 < 3000 ✗	TRAP: addressing error
0x4000	0x1000	0x02A4	0x02A4 < 0x1000 ✓	physical 0x42A4 (= 17060)
0x4000	0x1000	0x1200	0x1200 < 0x1000 ✗	TRAP: addressing error

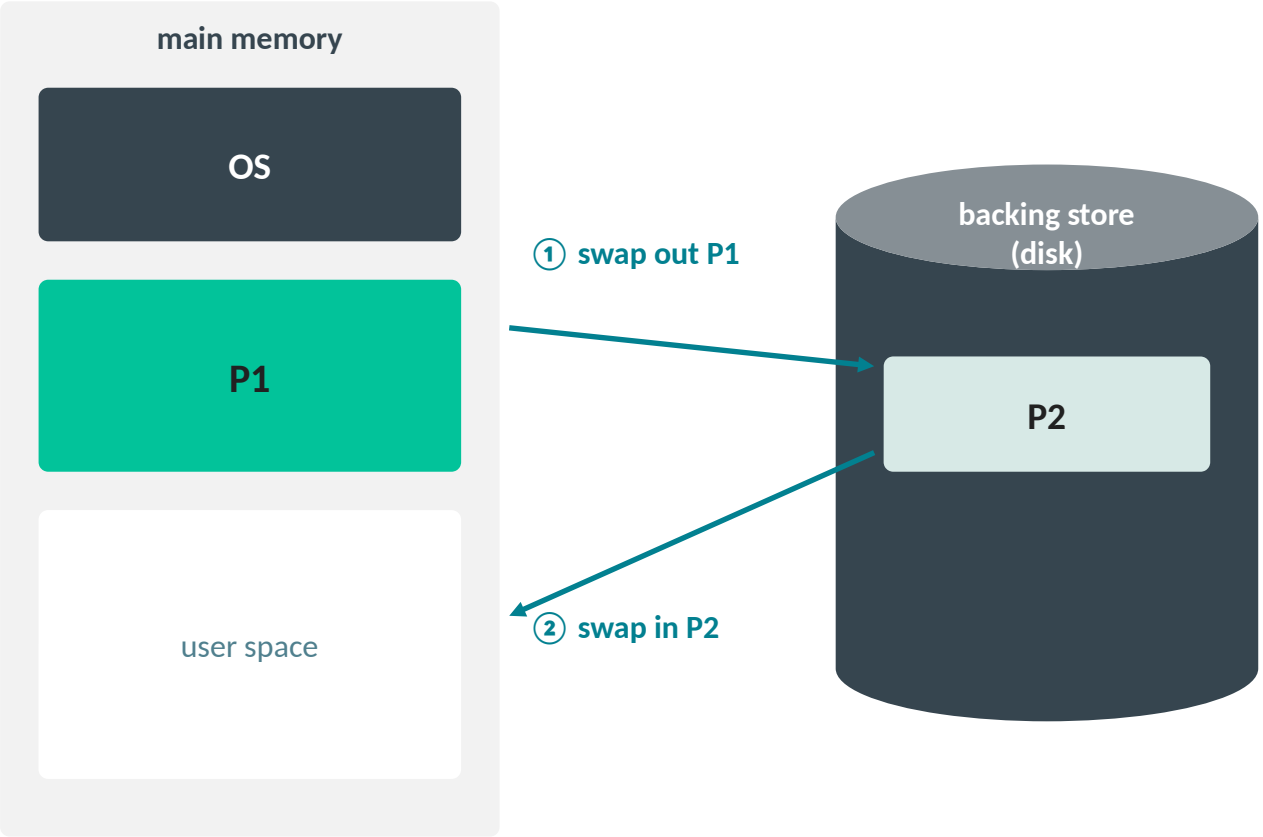
physical memory (decimal row): the process may touch only the mint range



The process owns exactly the physical range [base, base + limit): 14000 … 16999 in decimal, 0x4000 … 0x4FFF in hex (0x4000 = 16384, 0x1000 = 4096).

Swapping: Moving a Whole Process Out to Disk

If RAM is full, the OS can copy a process to a backing store (disk) and bring it back later.



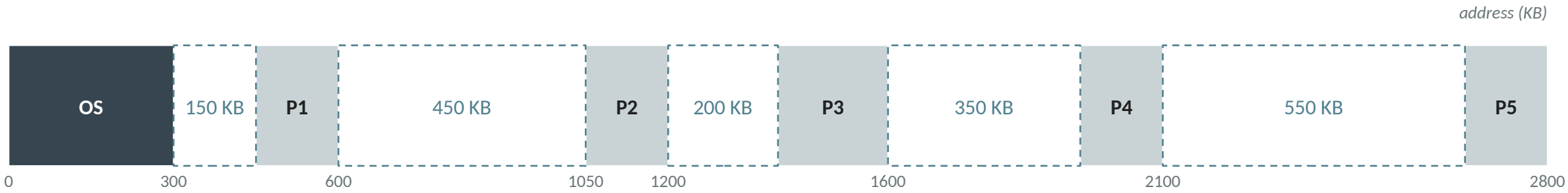

```
Process size     100 MB
Disk transfer     50 MB/s

swap out: 100 / 50 = 2 s
swap in : 100 / 50 = 2 s
total   ≈ 4 s   (plus disk latency)
```

4 s is 400 Round-Robin quanta of 10 ms! So modern systems do not swap whole processes. They swap single pages (4 KB ≈ 80 µs at 50 MB/s) — Week 7.

Variable Partitions and the Hole List

Each process gets one contiguous block of exactly the size it needs. Free blocks between processes are called holes.



Hole list kept by the OS

- (start, size) of every hole, here in address order:
- (300, 150) · (600, 450) · (1200, 200) · (1600, 350) · (2100, 550)
- Total free: 1700 KB, in 5 pieces.

What the OS does

- **Process arrives:** find a hole that is large enough, take what is needed, leave the rest as a smaller hole.
- **Process exits:** its block becomes a hole; merge it with neighbouring holes.

First-Fit, Best-Fit, Worst-Fit

Four processes arrive in this order: A = 220 KB, B = 400 KB, C = 130 KB, D = 430 KB. Which hole does each get?

First-fit

- Take the first hole (lowest address) that is big enough.
- Fast: the search can stop early.

Best-fit

- Take the smallest hole that is big enough.
- Leaves the smallest leftover; must search the whole list (or keep it sorted by size).

Worst-fit

- Take the largest hole.
- Idea: the leftover stays large enough to be useful. Must also search everything.

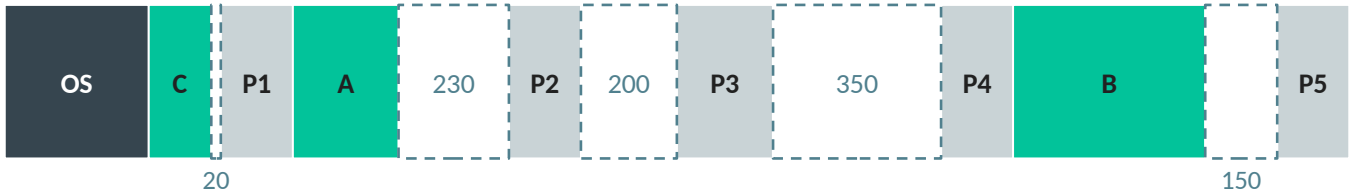
```
holes (size@start, KB):
150@300  450@600  200@1200  350@1600  550@2100

A = 220:  first → 450@600   best → 350@1600
          worst → 550@2100
```

Simulations (Silberschatz): first-fit and best-fit beat worst-fit in both time and storage use; first-fit is usually faster.

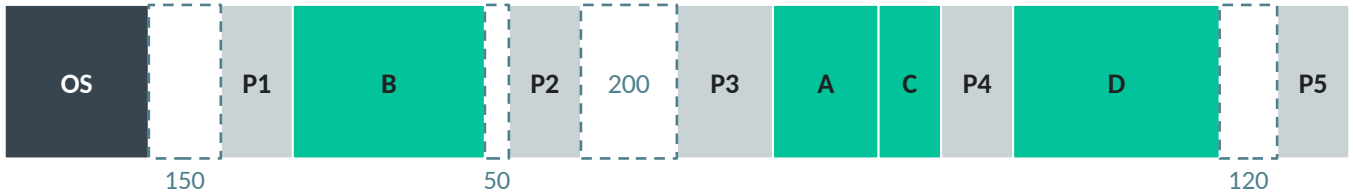
The Same Requests, Three Different Outcomes

First-fit



D must wait: 950 KB free, but no hole ≥ 430 KB.

Best-fit



All four fit.

Worst-fit



D must wait: 950 KB free, but no hole ≥ 430 KB.

new process existing process hole (size in KB)

Small holes (e.g. 20 KB) are labelled below the bar.

External vs. Internal Fragmentation

External fragmentation

- Enough free memory in total, but split into holes that are each too small.
- First-fit above: 950 KB free, D needs 430 KB, largest hole 350 KB.
- Typical of variable partitions and segmentation.
- **Cures:** compaction (next slide), or let a process be non-contiguous (paging, Part 2).

Internal fragmentation

- Memory given to a process but not used by it — waste inside the block.
- Happens when memory is handed out in fixed-size units: ask for 18,462 bytes, get 18,464.
- Typical of fixed partitions and paging.
- **Cure:** smaller allocation units — at the price of more bookkeeping (bigger tables).

50-percent rule (Silberschatz): with first-fit, for N allocated blocks about 0.5 N more are lost to fragmentation — up to one third of memory may be unusable.

Compaction: Shuffle Memory Together

Move all processes to one end so that all holes merge into one large hole.

Before (first-fit result)



After compaction: one hole of 950 KB → D (430 KB) fits



- **Possible only** with execution-time binding: change the relocation register after moving.

- **Expensive:** copies up to all of RAM; processes must pause, and I/O into a moving buffer is a problem.

INTERMISSION

10-Minute Break

Next: paging — the idea that removes external fragmentation.

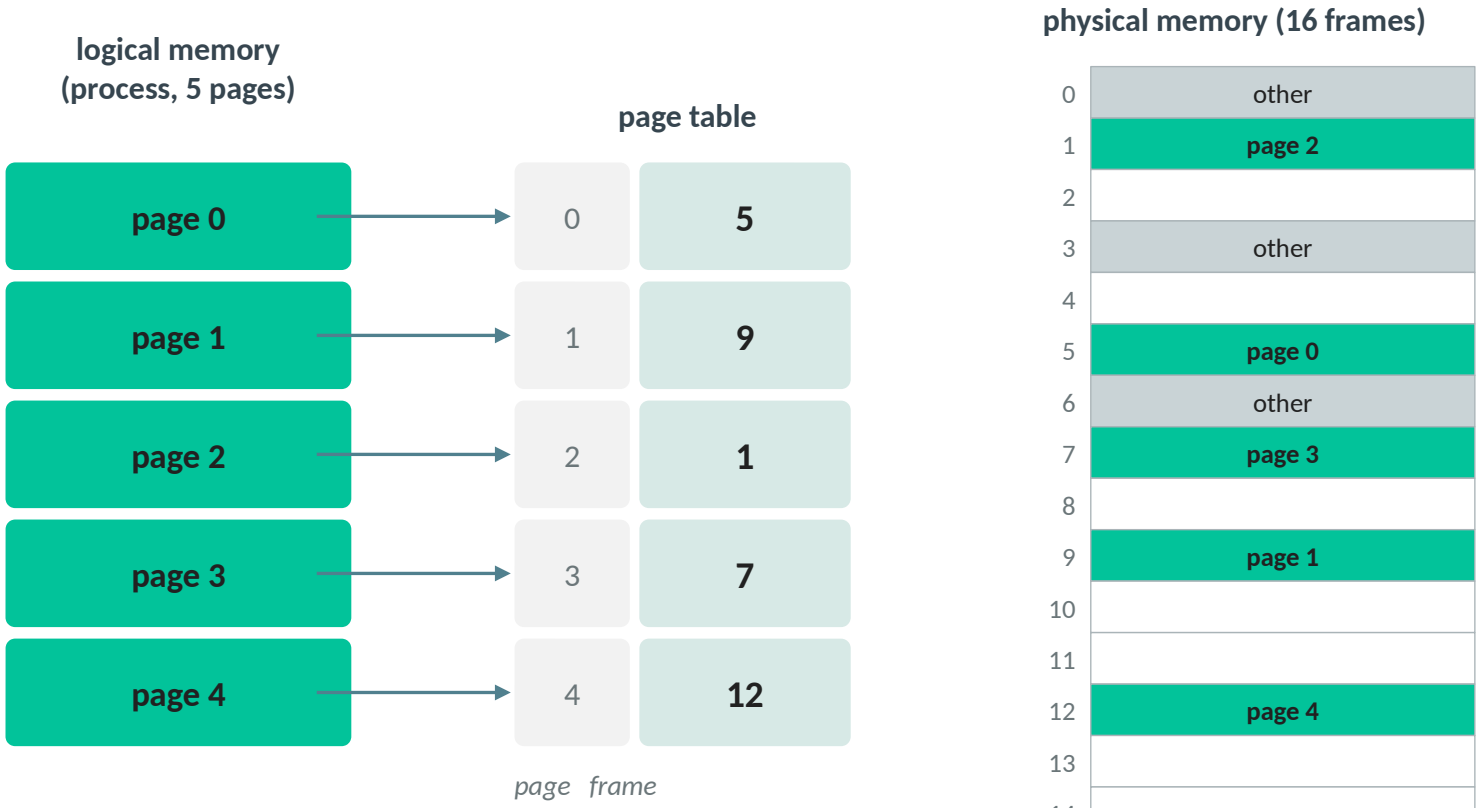
PART 2

Paging

Cut memory into equal pieces, and let a process live in pieces all over RAM.

Pages, Frames and the Page Table

Logical memory is cut into pages, physical memory into frames of the same size (here 4 KB). Any page can go into any free frame.



Key points

- Page size = frame size, a power of 2 (Linux/x86: 4 KB).
- A process's frames need not be contiguous or in order.
- Any free frame can be used → no external fragmentation.
- One page table per process; the OS keeps a free-frame list.

Splitting an Address into Page Number and Offset

With an m-bit logical address and page size 2^n bytes, the high m − n bits are the page number p and the low n bits the offset d.




```
p = address div 2^n   =  address >> n
d = address mod 2^n   =  address & (2^n - 1)
physical = frame(p) × 2^n + d

16-bit address, 4 KB pages (n = 12):  p = 4 bits, d = 12 bits
32-bit address, 4 KB pages (n = 12):  p = 20 bits, d = 12 bits

0x3A7F = 0011 | 1010 0111 1111
         p = 3   d = 0xA7F
```

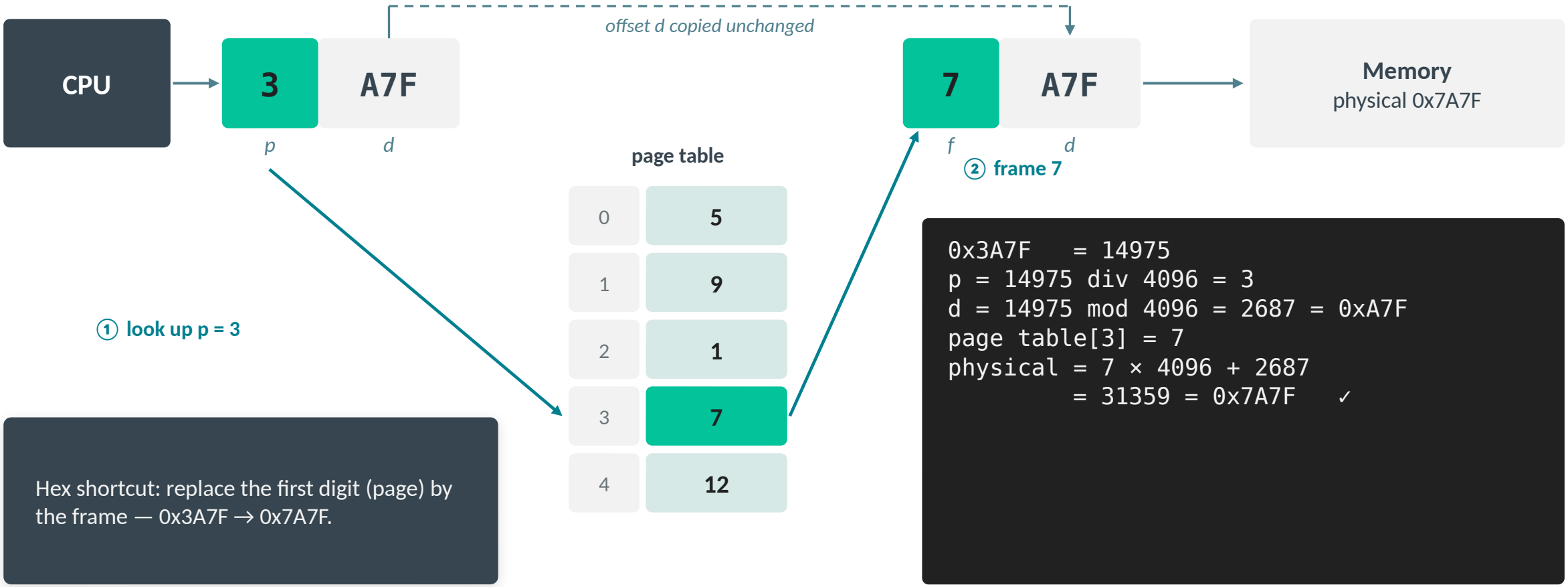
Hex shortcut

- One hex digit = 4 bits, so with 4 KB pages (12 bits) the offset is the last 3 hex digits.
- 0x3A7F → p = 0x3, d = 0xA7F.
- The offset is copied unchanged: only the page number is translated.

WORKED EXAMPLE

Translating Logical Address 0x3A7F

16-bit addresses, 4 KB pages (n = 12), page table from the previous slides.

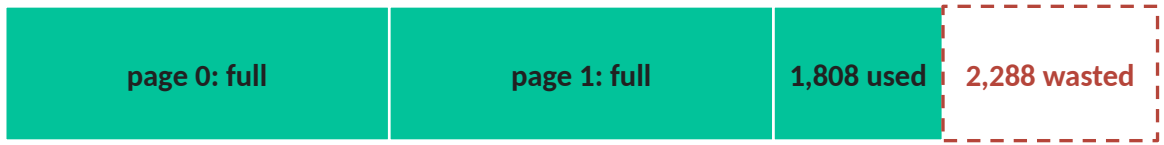


Internal Fragmentation and the Choice of Page Size

Paging has no external fragmentation — but the last page of a process is usually not full.

```
Process size   10,000 bytes
Page size       4,096 bytes

10,000 = 2 × 4,096 + 1,808
→ 3 frames, last frame holds 1,808 bytes
→ internal fragmentation = 2,288 bytes
```



frame boundaries every 4,096 bytes

Page size is a trade-off

- **Average waste:** half a page per process (2 KB for 4 KB pages); worst case one page minus one byte.
- **Small pages:** less waste, but more pages → bigger page tables.
- **Large pages:** smaller tables and more efficient disk I/O, but more waste.
- **Linux on x86-64:** 4 KB normally, plus "huge pages" of 2 MB or 1 GB for big programs such as databases.

How Big Is a Page Table?

One entry per page. With large address spaces the tables become very large.

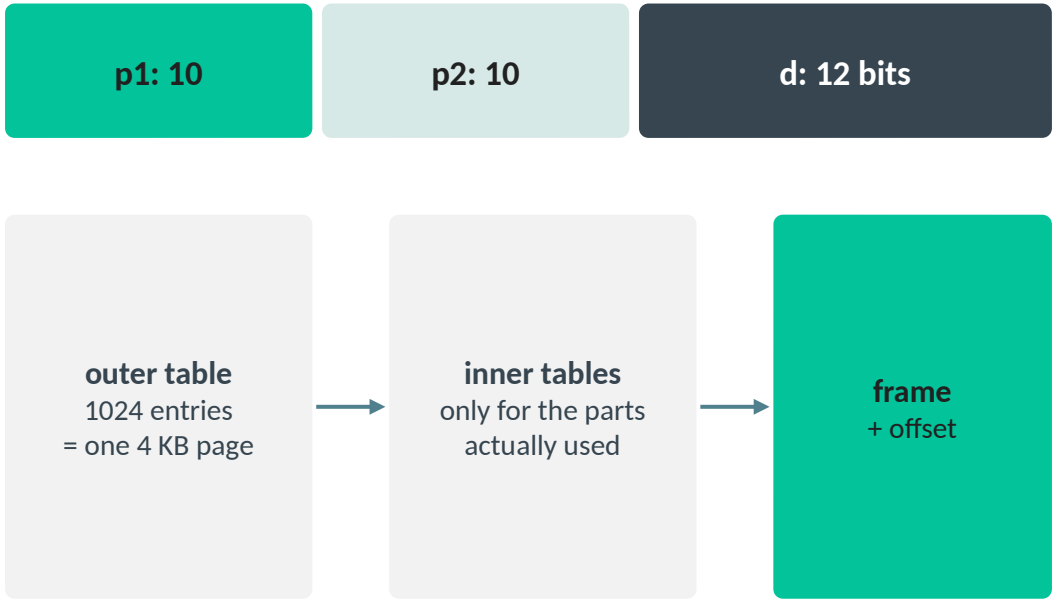
```
32-bit logical address, 4 KB pages, 4-byte entries

pages   = 2^32 / 2^12 = 2^20 = 1,048,576
size    = 2^20 × 4 B  = 4 MB   per process!

100 processes → 400 MB of page tables
```

Most of a process's address space is never used — so most of that table would be empty entries.

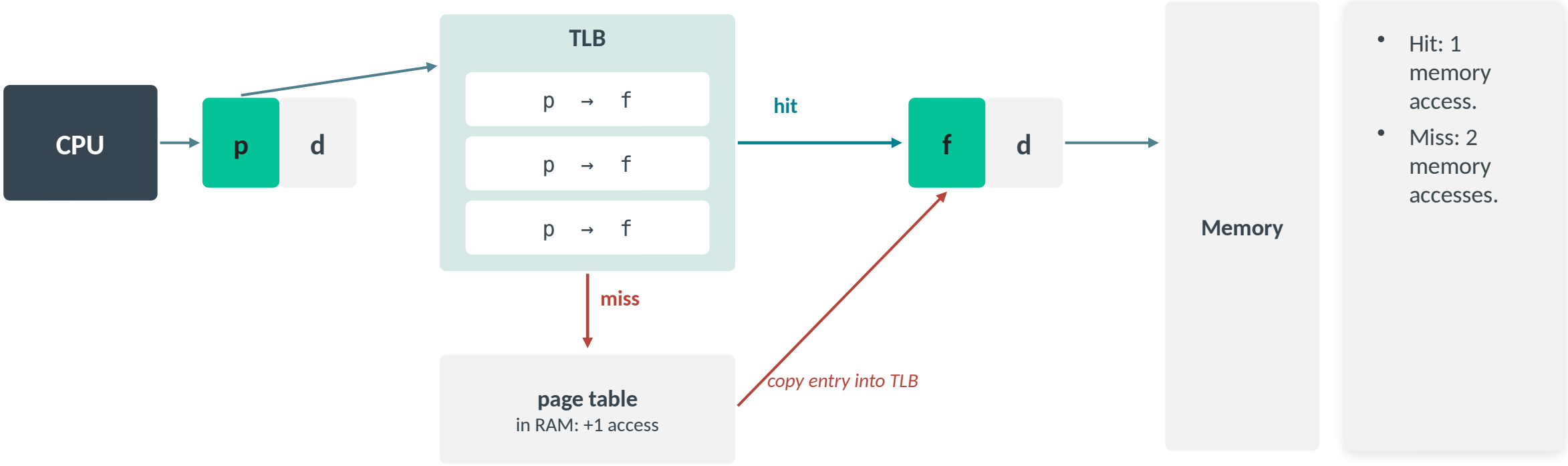
Two-level page table (32-bit)



The page table is itself paged. x86-64 uses 4 levels (9 + 9 + 9 + 9 + 12 = 48 bits); recent CPUs offer 5.

The TLB: A Cache for Page-Table Entries

The page table lives in RAM, so every access would need two memory accesses: one for the entry, one for the data.



TLB (translation look-aside buffer): small (≈ 64 – 1536 entries), very fast associative memory inside the MMU.

WORKED EXAMPLE

Effective Access Time (EAT)

Our convention: the TLB lookup time t_TLB is counted on every access; a miss costs one extra memory access.

$$\text{EAT} = \alpha\,(t_{TLB} + t_m) + (1-\alpha)(t_{TLB} + 2\,t_m)$$
$$= t_{TLB} + (2-\alpha)\,t_m \qquad \alpha = \text{TLB hit ratio}$$

$t_{TLB} = 20$ ns, $t_m = 100$ ns

$\alpha = 0.80$: $0.80 \times 120 + 0.20 \times 220 = 96 + 44 = 140$ ns (40 % slower)

$\alpha = 0.98$: $0.98 \times 120 + 0.02 \times 220 = 117.6 + 4.4 = 122$ ns (22 % slower)

no TLB: $2 \times 100 = 200$ ns (2× slower)

Reading the result

- Without a TLB, paging would double every memory access.
- The hit ratio matters most: 80 % → 98 % saves 18 ns per access.
- **Modern simplification** (Silberschatz 10e): the TLB lookup is overlapped with other work, so $t_{TLB} \approx 0$: EAT = $\alpha\, t_m + (1-\alpha)\, 2\, t_m$ → 120 ns for $\alpha = 0.80$, 101 ns for $\alpha = 0.99$.

Protection Bits and the Valid–Invalid Bit

Each page-table entry carries more than a frame number. The MMU checks the bits on every access.

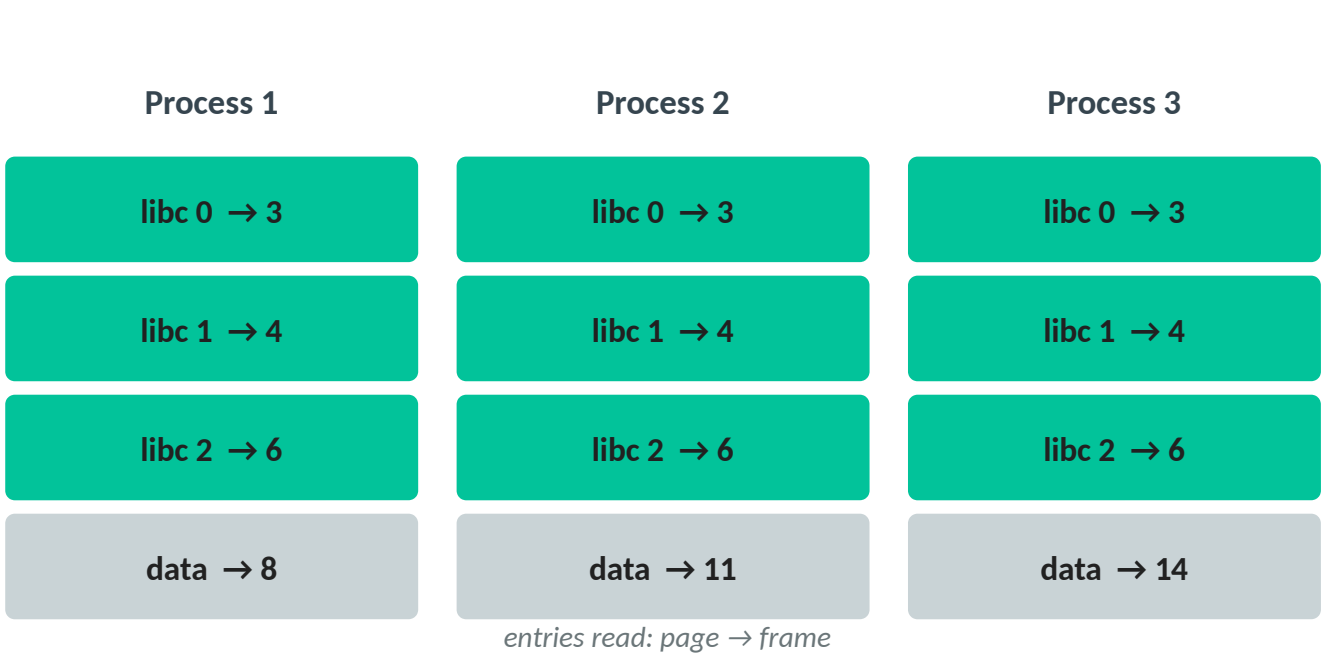
Page	Frame	Valid	Permissions	Contents
0	5	v	r-x	code
1	9	v	r-x	code
2	1	v	rw-	data
3	7	v	rw-	heap
4	12	v	rw-	stack
5 … 15	—	i	—	not part of the process

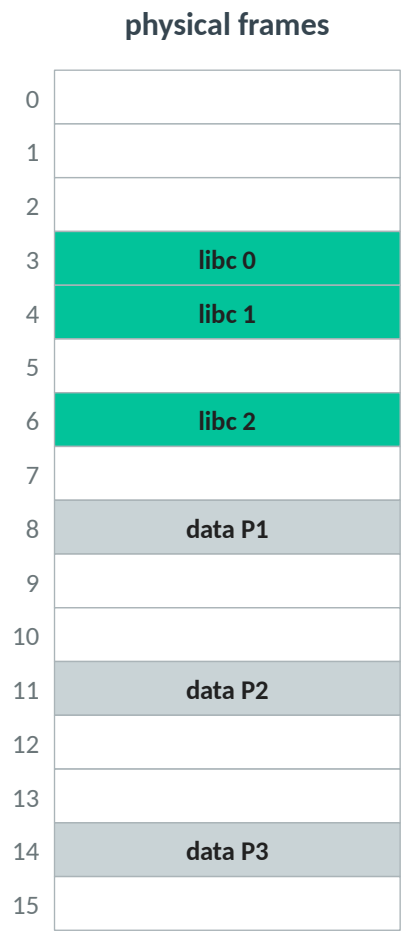
What the hardware checks

- **Valid bit = i:** the page is not in the process's address space → trap. Logical 0x5000 (page 5) traps.
- **Write to an r-x page:** trap. Code cannot be overwritten by accident.
- **Execute an rw- page:** trap (NX bit). Stops many attacks that inject code into data.
- You will see exactly these r, w, x letters in /proc/self/maps.

Shared Pages: One Copy of the C Library for Everyone

Read-only (reentrant) code can be mapped into many processes at once. Only the data pages are private.





- Three processes, but libc is in RAM only once.
- The shared pages are marked read-only.
- Same idea: fork() initially shares all pages (copy-on-write, Week 7).

With 40 processes and a 2 MB library, sharing saves 39 × 2 MB = 78 MB of RAM.

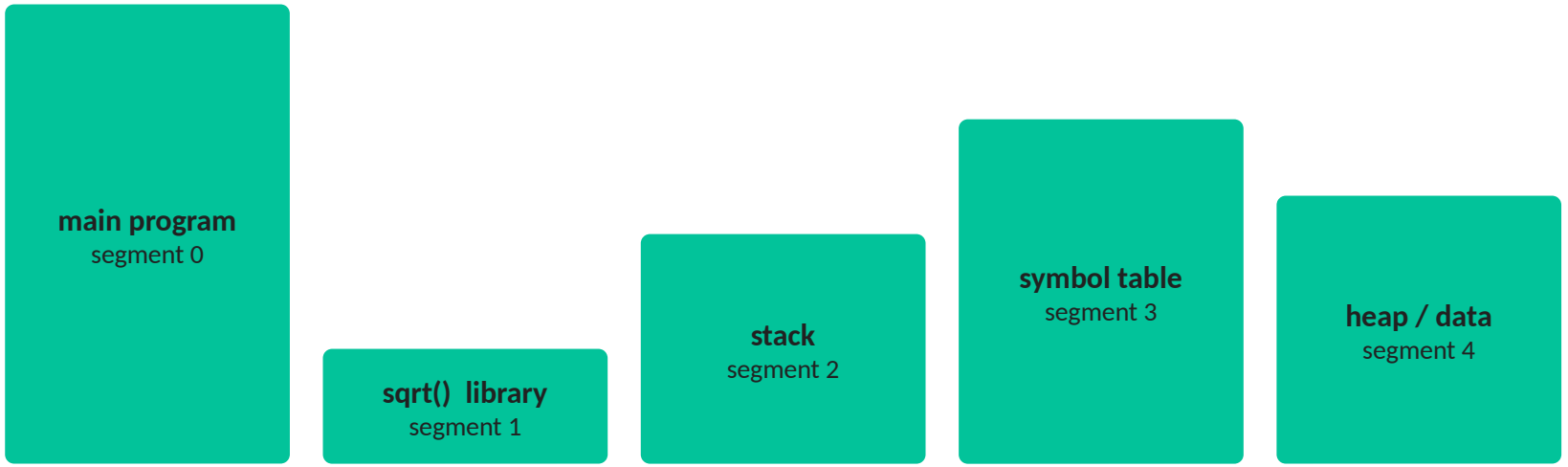
PART 3

Segmentation

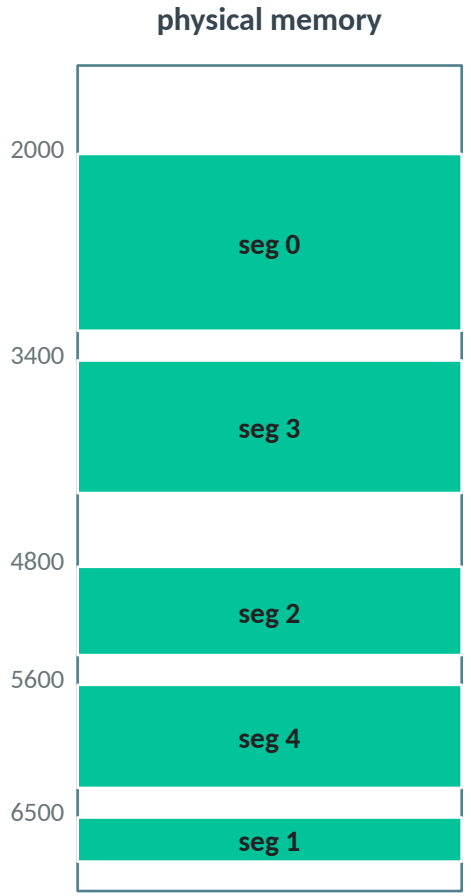
Memory as the programmer sees it: a collection of named parts of different sizes.

The User's View of a Program

A programmer does not think of 4 KB pages, but of a main program, functions, a stack, a symbol table, data …



logical view: separate pieces, each starting at offset 0, of different sizes

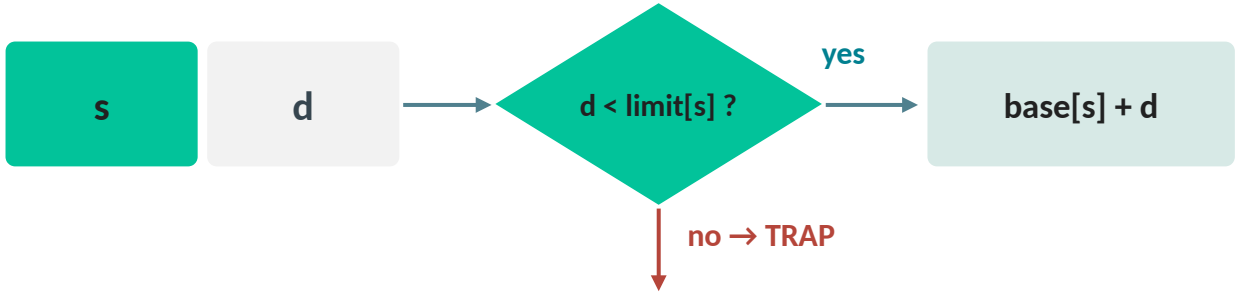


Logical address = ⟨segment number s, offset d⟩. The compiler creates the segments; the loader gives each one a base in physical memory.

Segment Table and Address Translation

Each segment has a base (where it starts in RAM) and a limit (its length). The offset is checked against the limit.

s	Segment	Base	Limit
0	main program	2000	1200
1	sqrt() library	6500	300
2	stack	4800	600
3	symbol table	3400	900
4	heap / data	5600	700




```
(s=2, d=53)  : 53 < 600    → 4800 + 53  = 4853
(s=3, d=852) : 852 < 900   → 3400 + 852 = 4252
(s=1, d=299) : 299 < 300   → 6500 + 299 = 6799
(s=0, d=1222): 1222 ≥ 1200 → TRAP
                (addressing error)
```

Same idea as base + limit — but with one pair per segment.

COMPARISON

Paging vs. Segmentation

	Paging	Segmentation
Unit size	fixed (e.g. 4 KB), chosen by hardware	variable, follows the program's structure
Visible to the programmer?	no — invisible	yes — main, stack, data …
Logical address	one number, split into p | d	a pair ⟨s, d⟩
External fragmentation	none	yes (variable-size blocks)
Internal fragmentation	yes, ½ page per process on average	none
Protection & sharing	per page	per meaningful unit (segment)
Table entry	frame number + bits	base + limit + bits

Combination: Multics and the Intel 80386 used segments made of pages. Modern 64-bit systems keep only paging.

REAL SYSTEMS

What x86-64 Linux Actually Does

Intel CPUs still contain segmentation hardware — but 64-bit operating systems essentially switch it off.

Segmentation: flat

- Code, data and stack segments have base 0 and cover the whole address space.
- Only FS/GS remain in use (for thread-local data).

Paging: everything

- 4-level page tables, 48-bit addresses (9+9+9+9+12); 5 levels / 57 bits on new CPUs.
- Pages of 4 KB, 2 MB, 1 GB.

Per-process tables

- Each process has its own page tables; register CR3 points to them.
- A context switch reloads CR3 (Week 3).

The segment idea lives on in software: Linux keeps a list of regions (text, heap, stack, libraries) per process — /proc/self/maps shows it.

HANDS-ON

Example 1: Where Do My Variables Live?

```c
int initialised_global = 42;   /* data segment            */
int uninitialised_global;      /* bss segment (zero-filled) */

int main(void)
{
    int local = 7;                        /* stack */
    int *dynamic = malloc(sizeof(int));   /* heap  */

    printf("text  (code)   main()      : %p\n", (void *)main);
    printf("data  (global) initialised : %p\n",
           (void *)&initialised_global);
    printf("bss   (global) zero-filled : %p\n",
           (void *)&uninitialised_global);
    printf("heap  (malloc)             : %p\n", (void *)dynamic);
    printf("stack (local variable)     : %p\n", (void *)&local);
    printf("page size                  : %ld bytes\n",
           sysconf(_SC_PAGESIZE));
    show_maps();   /* print our lines of /proc/self/maps */
    free(dynamic);
}
/* full program: code/mem_layout.c */
```

Observe

- Code and globals: low addresses (0x55…).
- Heap: a bit higher; stack: near the top (0x7ff…).
- Each address lies in a region of /proc/self/maps with matching permissions.

```
$ ./mem_layout
text (code) main() : 0x56412f3601e0
data (global) initialised : 0x56412f363010
bss (global) zero-filled : 0x56412f363018
heap (malloc) : 0x564160c6c2a0
stack (local variable) : 0x7ffe905b48c4
page size                  : 4096 bytes
56412f35f000-56412f360000 r--p mem_layout
56412f360000-56412f361000 r-xp mem_layout
56412f361000-56412f362000 r--p mem_layout
56412f362000-56412f363000 r--p mem_layout
56412f363000-56412f364000 rw-p mem_layout
564160c6c000-564160c8d000 rw-p [heap]
7ffe90595000-7ffe905bb000 rw-p [stack]
```

HANDS-ON

The Page Size and the Memory Map from the Shell

```
$ getconf PAGESIZE
4096
$ grep Hugepagesize /proc/meminfo
Hugepagesize:       2048 kB
$ head -n 6 /proc/self/maps        # the memory map of the head process itself
559948881000-559948883000 r--p 00000000 fe:00 270606         /usr/bin/head
559948883000-559948889000 r-xp 00002000 fe:00 270606         /usr/bin/head
559948889000-55994888b000 r--p 00008000 fe:00 270606         /usr/bin/head
55994888b000-55994888c000 r--p 00009000 fe:00 270606         /usr/bin/head
55994888c000-55994888d000 rw-p 0000a000 fe:00 270606         /usr/bin/head
559970df7000-559970e18000 rw-p 00000000 00:00 0              [heap]
```

Columns

start–end (virtual), permissions r/w/x + p(rivate) or s(hared), file offset, device, inode, file.

Page arithmetic

0x559948883000 − 0x559948881000 = 0x2000 = 8192 bytes = 2 pages. Every boundary ends in 000: regions are page-aligned.

Same file, 5 regions

One program file, mapped with different permissions: headers, code (r-x), constants, data (rw-).

HANDS-ON

Example 2: Same Address, Different Values

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int x = 100;                    /* one global variable */

int main(void)
{
    pid_t pid = fork();         /* now there are two processes */

    if (pid == 0) {             /* ---- child ---- */
        x = 200;                /* change OUR copy only       */
        printf("child  (PID %d): &x = %p   x = %d\n",
               getpid(), (void *)&x, x);
        return 0;
    }
    wait(NULL);                 /* ---- parent: wait for the child */
    printf("parent (PID %d): &x = %p   x = %d\n",
           getpid(), (void *)&x, x);
    return 0;
}
```

Why is this possible?

- fork() copies the parent: same layout, same addresses.
- The child sets x = 200; the parent still sees 100.
- &x is a logical address. Each process has its own page table, mapping it to a different frame.

```
$ ./fork_vaddr
child  (PID 1760): &x = 0x55bc4a29c010  x = 200
parent (PID 1759): &x = 0x55bc4a29c010  x = 100
```

Example 3: Page Number and Offset with Bit Operations

```
#define N_BITS    12                 /* page size 2^12 = 4096 */
#define PAGE_SIZE (1UL << N_BITS)
#define OFF_MASK  (PAGE_SIZE - 1)    /* 0xFFF                 */

/* page -> frame; -1 = invalid page */
int page_table[16] = { 5, 9, 1, 7, 12, -1, /* ... */ };

for (int i = 1; i < argc; i++) {
    unsigned long a = strtoul(argv[i], NULL, 0);
    unsigned long p = a >> N_BITS;      /* high bits  */
    unsigned long d = a & OFF_MASK;     /* low 12 bits */
    if (p >= 16 || page_table[p] < 0) continue;  /* trap */
    unsigned long f = page_table[p];
    unsigned long phys = (f << N_BITS) | d;   /* f*4096 + d */
    printf(... a, p, d, f, phys);
}
/* full program: code/page_split.c */
```

What it does

- p = a >> 12 and d = a & 0xFFF.
- Translation: (frame << 12) | d.
- Same page table as on the slides.

```
$ ./page_split 0x3A7F 0x1234 0x5000
0x3A7F(14975):
  p= 3 d=0xA7F(2687)→frame  7→0x7A7F(31359)
0x1234( 4660):
  p= 1 d=0x234( 564)→frame  9→0x9234(37428)
0x5000(20480):
  p= 5 d=0x000(   0)→invalid: trap to OS
```

TRY IT YOURSELF

Explore Memory on Your Own Machine

```
# Linux (lab PCs, WSL, or a virtual machine)
$ getconf PAGESIZE          # page size in bytes
$ cat /proc/self/maps       # memory map of cat
$ ./mem_layout; ./mem_layout   # run twice: ASLR
$ free -h                   # RAM and swap in use
$ cat /proc/swaps           # swap areas
$ vmstat 1 5                # si/so = swap in/out

# Windows
# Task Manager > Performance > Memory:
# "In use", "Committed", "Paged pool"
```

Questions to answer

- What is the page size on your machine? On a Mac with Apple silicon, try it: 16 KB.
- How many pages large is the [stack] region of cat?
- Do the addresses printed by mem_layout change between runs? Why might that be good for security?
- Is your machine using swap right now? How much?

QUICK QUIZ

Check Your Understanding

1. Base = 0x2000, limit = 0x0800. Where does logical 0x07FF go? And logical 0x0800?

2. Pages are 4 KB. Split logical address 0x5C21 into page number p and offset d.

3. t_TLB = 20 ns, t_m = 100 ns, hit ratio 90 %. Compute the effective access time.

4. Why does paging have no external fragmentation — but still internal fragmentation?

Week 6 — Key Takeaways

- **Logical vs. physical:** programs use logical addresses; the MMU translates them on every access (execution-time binding).
- **Base + limit:** the simplest MMU — check logical < limit, then add base; otherwise trap.
- **Contiguous allocation:** first-, best-, worst-fit; external fragmentation (50-% rule), compaction is costly.
- **Paging:** pages ↔ frames, address = p | d, page table per process; no external, only internal fragmentation.
- **TLB:** caches page-table entries; EAT = t_TLB + (2 − α) t_m with our convention.
- **Segmentation:** ⟨segment, offset⟩ with base/limit per segment; x86-64 Linux uses paging with flat segments.

NEXT WEEK

Virtual Memory & File Systems

Week 7 will cover:

- Demand paging and page faults
- Page replacement: FIFO, OPT, LRU — and thrashing
- File systems: the file concept, access methods, directory structures
- Last lecture before the midterm (Week 8 covers Weeks 1–7)

QUESTIONS?

Discussion

If you have questions after class, ask them on the course Telegram channel — everyone benefits from the answer.

Asst. Prof. Dr. Hasan Oğuz